

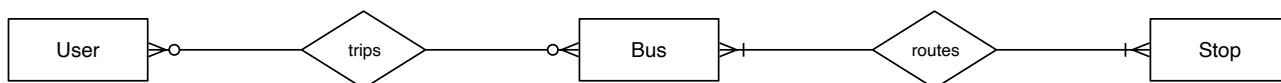
TITSA

TITSA (Transportes Interurbanos de Tenerife S.A.) es la empresa que se dedica al transporte en **guagua** de la ciudadanía de Tenerife.

El objetivo es desarrollar una aplicación de **gestión de guaguas** (*nombre canario para “autobus”*) en línea de comandos usando el paradigma de *Programación Orientada a Objetos* con Python junto al módulo **sqlite** para el *acceso a datos*.

1. Base de datos

El ERD (Esquema Entidad-Relación) de la base de datos con la que vamos a trabajar es el siguiente:



- Un usuario puede tomar 0 o muchas guaguas; Una guagua puede llevar a 0 o muchos usuarios.
- Una guagua puede pasar por 1 o muchas paradas; Una parada puede contemplar 1 o muchas guaguas.

1.1. Tabla users

Almacena usuarios (*viajeros*) de TITSA.

Campos	Descripción
id	Identificador del usuario (DNI) PK
name	Nombre y apellidos
credit	Saldo para viajar (en euros)

```

CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    credit REAL DEFAULT 0
);
  
```

Ejemplo de filas:

65476521Y|Guido van Rossum|126

1.2. Tabla buses

Representa guaguas de TITSA.

Campos	Descripción
id	Identificador de la guagua (número de línea) PK
total_seats	Número de asientos totales de la guagua
busy_seats	Número de asientos ocupados de la guagua

```
CREATE TABLE IF NOT EXISTS buses (  
    id TEXT PRIMARY KEY,  
    total_seats INTEGER NOT NULL,  
    busy_seats INTEGER NOT NULL DEFAULT 0,  
    CHECK (total_seats >= 0),  
    CHECK (busy_seats >= 0),  
    CHECK (busy_seats <= total_seats)  
);
```

Ejemplo de filas:

102|60|21

1.3. Tabla stops

Representa paradas en la red de transporte de TITSA.

Campos	Descripción
id	Identificador de la parada (número de parada) PK
name	Nombre de la parada
cost	Coste para el usuario de “usar” esta parada

```
CREATE TABLE IF NOT EXISTS stops (  
    id TEXT PRIMARY KEY,  
    name TEXT NOT NULL,  
    cost REAL DEFAULT 0  
);
```

Ejemplo de filas:

1281|NUEVO OBRERO|0.13
4095|TAORO|0.85

1.4. Tabla trip

Representa la relación user-bus donde el usuario **viaja** en una guagua.

Campos	Descripción
user_id	Identificador del usuario
bus_id	Identificador de la guagua
stop_id	Identificador de la parada donde el usuario sube/baja de la guagua
moment	Momento en el que el usuario sube/baja de la guagua
status	'IN' si el usuario se sube a la guagua; 'OUT' si el usuario se baja de la guagua

```
CREATE TABLE IF NOT EXISTS trip (
    user_id TEXT NOT NULL,
    bus_id TEXT NOT NULL,
    stop_id TEXT NOT NULL,
    moment DATETIME NOT NULL,
    status TEXT NOT NULL,

    PRIMARY KEY (user_id, bus_id, moment)

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (bus_id) REFERENCES buses(id),
    FOREIGN KEY (stop_id) REFERENCES stops(id)
);
```

Ejemplo de filas:

65476521Y|102|1281|2026-03-26 15:30:45|IN

65476521Y|102|4095|2026-03-26 15:56:21|OUT

1.5. Tabla route

Representa la relación bus-stop donde la guagua **transita** por una parada.

Campos	Descripción
bus_id	Identificador de la guagua
stop_id	Identificador de la parada
order	Orden de la parada en el trayecto de la guagua

```
CREATE TABLE IF NOT EXISTS route (
    bus_id TEXT NOT NULL,
    stop_id TEXT NOT NULL,
    stop_order INTEGER NOT NULL,

    PRIMARY KEY (bus_id, stop_id)

    FOREIGN KEY (bus_id) REFERENCES buses(id),
    FOREIGN KEY (stop_id) REFERENCES stops(id)
);
```

Ejemplo de filas:

102|1281|4

102|4095|21

2. Implementación

Será necesaria la implementación de los siguientes módulos dentro del paquete `src`:

- `settings.py`
- `db.py`
- `errors.py`
- `helpers.py`
- `users.py`
- `stops.py`
- `buses.py`

Recuerda utilizar los “imports” de esta manera cuando vayas a testear tu código:

```
from .db import DbHandler
from .errors import TitsaError
from .settings import DB_PATH, LOG_PATH
from .stops import Stop
from .users import User
```

2.1. `settings.py`

Este módulo contiene las configuraciones relevantes del proyecto.

2.2. `db.py`

Este módulo contiene los artefactos necesarios para el manejo de la base de datos.

2.2.1. Clase `DbHandler`

```
def __init__(self, db_path: str = DB_PATH)
```

- Se debe crear el atributo de instancia `con` (*conexión*), indicando que trabaja en modo “autocommit” y con resultados en modo “row”.
- Se debe crear el atributo de instancia `cur` (*cursor*).

```
def run_sql(self, query: str, **params)
```

- Debe ejecutar la `query` (*consulta*) con los parámetros `params`.
- Si la consulta es de tipo `SELECT` habrá que recuperar todos los resultados de la consulta y devolverlos. Ojo porque la consulta puede venir en minúsculas o en mayúsculas.
- En otro caso no es necesario devolver nada.

2.3. errors.py

Este módulo se encarga de la gestión de errores del proyecto.

2.3.1. Clase TitsaError

```
def __init__(self, message: str)
```

- Se debe construir una excepción desde la correspondiente herencia y el mensaje que se pasa por parámetro.
- Se debe guardar el mensaje en el fichero de *log* (modo “append”) definido en `settings.py`

2.4. helpers.py

Este módulo contiene funciones auxiliares necesarias en el proyecto.

```
def create_db(db_path: str = ':memory:') -> None:
```

- Se debe instanciar la clase `DbHandler` con la ruta `db_path`.
- A través de dicha instancia se deben crear las siguientes tablas en la base de datos:
 - `users`
 - `buses`
 - `stops`
 - `trip`
 - `route`
- El código SQL correspondiente a cada tabla lo tienes en el primer apartado del documento.

2.5. users.py

Este módulo se encarga de la gestión de usuarios (*viajeros*).

2.5.1. Clase User

Esta clase debe heredar de `DbHandler` para poder hacer uso de sus métodos en la interacción con la base de datos.

```
def __init__(self, id: str, name: str = '', credit: float = 0):
```

- Si el `id` de usuario (DNI) no casa con la expresión regular de un DNI entonces se debe elevar una excepción de tipo `TitsaError` con el mensaje `Invalid user ID format: YY432W`
- Recuerda invocar al constructor de la clase base.
- Se debe crear un atributo de instancia `id` correspondiente al parámetro `id` que se pasa por parámetro.
- Se debe crear el usuario en la tabla `users`.
- Si el usuario ya existía en la base de datos, no hay que volverlo a crear.

```
def name(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el nombre del usuario `self`
- Si el usuario no existe devuelve cadena vacía.

```
def name(self, value: str) -> None:
```

- Decora con `@name.setter`
- Actualiza la base de datos con el nuevo nombre del usuario `self`

```
def credit(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el crédito del usuario `self`
- Si el usuario no existe devuelve 0.0

```
def credit(self, value: float) -> None:
```

- Decora con `@credit.setter`
- Actualiza la base de datos con el nuevo crédito del usuario `self`

2.6. stops.py

Este módulo se encarga de la gestión de paradas de guaguas.

2.6.1. Clase Stop

Esta clase debe heredar de `DbHandler` para poder hacer uso de sus métodos en la interacción con la base de datos.

```
def __init__(self, id: str, name: str = '', cost: float = 0):
```

- Si el id de la parada **compuesto únicamente por 4 dígitos** entonces se debe elevar una excepción de tipo `TitsaError` con el mensaje **Invalid stop ID format: W342**
- Si el coste `cost` (en euros) por usar la parada no está entre 0 y 1 (incluidos) se debe elevar una excepción de tipo `TitsaError` con el mensaje **Invalid stop cost. Must be between 0 and 1: 4.67** (el valor 4.67 es un ejemplo).
- Recuerda invocar al constructor de la clase base.
- Se debe crear un atributo de instancia `id` correspondiente al parámetro `id` que se pasa por parámetro.
- Se debe crear la parada en la tabla `stops`.
- Si la parada ya existía en la base de datos, no hay que volver a crearla.

```
def name(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el nombre de la parada `self`
- Si la parada no existe devuelve cadena vacía.

```
def name(self, value: str) -> None:
```

- Decora con `@name.setter`
- Actualiza la base de datos con el nuevo nombre de la parada `self`

```
def cost(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el coste por usar la parada `self`
- Si la parada no existe devuelve 0.0

```
def cost(self, value: str) -> None:
```

- Decora con `@cost.setter`
- Actualiza la base de datos con el nuevo coste por usar la parada `self`

```
def load_stops_from_file(cls, filename: str) -> None:
```

- Decora con `@classmethod`
- Este método lee un fichero y carga paradas en la base de datos.
- Cada línea del fichero tiene el formato `4182,EL CASINO,0.23` donde el primer valor es el identificador de la parada, el segundo valor es el nombre de la parada y el tercer valor es el coste por usar la parada.
- Recuerda que por cada línea del fichero se debe crear una nueva parada en la base de datos (el constructor ya se encarga de esto).

2.7. buses.py

Este módulo se encarga de la gestión de guaguas.

2.7.1. Clase Bus

Esta clase debe heredar de `DbHandler` para poder hacer uso de sus métodos en la interacción con la base de datos.

```
def __init__(self, id: str, total_seats: int = 0):
```

- Si el id de la guagua no está **compuesto únicamente por 2 o 3 dígitos** entonces se debe elevar una excepción de tipo `TitsaError` con el mensaje `Invalid bus ID format: A5488T`
- Recuerda invocar al constructor de la clase base.
- Se debe crear un atributo de instancia `id` correspondiente al parámetro `id` que se pasa por parámetro.
- Se debe crear la guagua en la tabla `bus`.
- Si la guagua ya existía en la base de datos, no hay que volver a crearla.
- Los asientos ocupados (`busy_seats`) inicialmente son 0.

```
def total_seats(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el número total de asientos de la guagua `self`
- Si la guagua no existe devuelve 0.

```
def total_seats(self, value: str) -> None:
```

- Decora con `@total_seats.setter`
- Actualiza la base de datos con el nuevo número de asientos de la guagua `self`

```
def busy_seats(self) -> str:
```

- Decora con `@property`
- Consulta en la base de datos y devuelve el número total de asientos ocupados de la guagua `self`
- Si la guagua no existe devuelve 0.

```
def busy_seats(self, value: str) -> None:
```

- Decora con `@busy_seats.setter`
- Actualiza la base de datos con el nuevo número de asientos ocupados de la guagua `self`

```
def add_stop(self, stop: Stop, stop_order: int) -> None:
```

- Inserta en la base de datos una nueva parada en el trayecto de la guagua `self`
- `stop` representa la parada y `stop_order` el orden de la parada dentro del trayecto de la guagua `self` (empezando siempre desde 1).

```
def load_route_from_file(self, filename: str) -> None:
```

- Este método lee un fichero y carga el trayecto (*paradas*) de la guagua `self` en la tabla `route`.
- Cada línea del fichero tiene el formato `353,6,4179` donde el primer valor es el identificador de la guagua, el segundo valor es el orden de la parada en el trayecto de la guagua y el tercer valor es el identificador de la parada.
- Recuerda que por cada línea del fichero se debe crear una nueva parada en la base de datos vinculada al trayecto de esta guagua.
- Aprovecha el método `add_stop()` para ello.

```
def is_full(self) -> bool:
```

- Decora con `@property`
- Devuelve si la guagua está llena o no.

```
def first_stop(self) -> Stop | None:
```

- Decora con `@property`
- Devuelve la primera parada del trayecto de la guagua `self`
- Devuelve `None` si el trayecto no tiene paradas.

```
def last_stop(self) -> Stop | None:
```

- Decora con `@property`
- Devuelve la última parada del trayecto de la guagua `self`
- Devuelve `None` si el trayecto no tiene paradas.

```
def get_stops(self, from_stop: Stop | None = None, to_stop: Stop | None = None):
```

- **Función generadora** que devuelve las paradas del trayecto de la guagua `self` entre la parada `from_stop` y la parada `to_stop`.
- Si el parámetro `from_stop` es `None` se considerará como inicio la primera parada del trayecto.
- Si el parámetro `to_stop` es `None` se considerará como final la última parada del trayecto.
- Recuerda que lo que se debe devolver son instancias de la clase `Stop`

```
def user_is_on_bus(self, user: User) -> bool:
```

- Indica si el usuario `user` se encuentra ahora mismo en la guagua `self`

```
def user_last_trip_cost(self, user: User) -> float:
```

- Devuelve el coste total del último viaje del usuario `user` en la guagua `self`
- El coste total se calcula como la suma de los costes de cada parada por las que pasó el usuario.
- Se recomienda utilizar el método `get_stops()` que devuelve las paradas entre dos puntos.
- Si en el último viaje realizado por el usuario hay menos de dos paradas almacenadas, se debe devolver 0.0

```
def user_trip(self, user: User, stop: Stop, status: str) -> None:
```

- Inserta en la tabla `trip` que el usuario `user` se ha subido o bajado a la guagua `self` en la parada `stop`
- Puedes utilizar `CURRENT_TIMESTAMP` para el momento.
- `status` puede ser `'IN'` o `'OUT'`.
- Este método no se testea. Es únicamente una propuesta de refactorización para llamarla desde los siguientes métodos `user_get_on()` y `user_get_off()`.

```
def user_get_on(self, user: User, stop: Stop) -> None:
```

- Refleja en la base de datos que el usuario `user` se ha subido a la guagua `self` en la parada `stop`.
- Si el usuario ya estaba en la guagua, habrá que elevar una excepción de tipo `TitsaError` con el mensaje `User 12345678A is already on bus 353`
- Si la guagua está llena, habrá que elevar una excepción de tipo `TitsaError` con el mensaje `Bus 353 is full`
- Se debe actualizar el número de asientos ocupados en la guagua `self` (recuerda los *setters*)

```
def user_get_off(self, user: User, stop: Stop) -> None:
```

- Refleja en la base de datos que el usuario `user` se ha bajado de la guagua `self` en la parada `stop`.
- Si el usuario no estaba en la guagua, habrá que elevar una excepción de tipo `TitsaError` con el mensaje `User 12345678A is not on bus 353`
- Se debe actualizar el número de asientos ocupados en la guagua `self` (recuerda los *setters*)
- Se debe actualizar el crédito del usuario. Para ello utiliza el método `user_last_trip_cost()` explicado previamente.